Profundidad por recurrencia: un *Looped Transformer* frente a GPT en problemas aritméticos en castellano de dificultad controlada

Alejandro García Belmonte
Universitat d'Alacant

Stanislav Gatin
Universitat d'Alacant

Rodrigo Gavilán Salmerón
Universitat d'Alacant

Pau Mateo Lillo
Universitat d'Alacant

Resumen

Un *Looped Transformer* aplica N veces un único bloque con los mismos pesos y obtiene profundidad de cómputo sin parámetros adicionales. Lo implementamos desde cero junto a un GPT, con una interfaz común validada numéricamente, y los entrenamos en problemas aritméticos en castellano de 1 a 6 operaciones, reservando los niveles 5 y 6 para medir la generalización. Comparamos ambas familias a igual profundidad, parámetros y cómputo, y estudiamos el interior del bucle con *logit lens* y métricas de representación y atención. Con ocho iteraciones, un Looped Transformer de 0.97 M de parámetros iguala o supera a un GPT de ocho capas con 6.5 M, sobre todo en problemas con más operaciones que los vistos al entrenar, y las iteraciones necesarias crecen con la dificultad; con dos y cuatro queda al nivel de un GPT de igual profundidad o algo por debajo. Código: https://github.com/rodrigogavsal/looped-math.

1. Introducción

Los Transformers resuelven los problemas de varios pasos aplicando una sucesión de capas, cada una con sus propios parámetros. La profundidad parece clave para el razonamiento (Saunshi et al., 2025), pero añadir capas encarece de forma lineal los parámetros, la memoria y el entrenamiento. El Universal Transformer (Dehghani et al., 2019) ya aplica de forma recurrente un mismo bloque; trabajos recientes lo estudian como *Looped Transformer*: un único bloque aplicado N veces sobre su propia salida, con profundidad de cómputo creciente y parámetros constantes (Giannou et al., 2023; Saunshi et al., 2025; Geiping et al., 2025).

Estudiamos esta propuesta a pequeña escala y con control total de los datos: entrenamos desde cero GPT y Looped Transformers de pocos millones de parámetros en problemas aritméticos en castellano cuya dificultad es el número de operaciones. Buscamos responder a **(P1)** si un bucle de N iteraciones iguala a un GPT de N capas con hasta N veces menos parámetros (al compartir el bloque), **(P2)** si las iteraciones necesarias crecen con la dificultad y **(P3)** si el modelo generaliza a problemas con más operaciones que los vistos al entrenar. Además, **(P4)** analizamos el interior del bucle con *logit lens* y métricas de representación y atención.

Aportamos una implementación propia, sobre el código de Raschka (2024), de ambos modelos con una interfaz común y validada numéricamente (Sec. 3.2); un conjunto de problemas sintéticos con seis niveles y los dos más difíciles reservados para evaluación (Sec. 3.3); y una comparación controlada a igual cómputo y a igual número de parámetros, con barrido de iteraciones y análisis interno del bucle (Secs. 4 y 5).

2. Trabajo relacionado

Pesos compartidos y bucles. El Universal Transformer (Dehghani et al., 2019) aplica de forma recurrente un mismo bloque sobre la secuencia. Giannou et al. (2023) muestran que un Transformer con bucles puede ejecutar algoritmos como un ordenador programable, y Saunshi et al. (2025) comparan, a igual profundidad efectiva, modelos con y sin bucles en tareas sintéticas de razonamiento. A mayor escala, Geiping et al. (2025) aumentan el cómputo en inferencia iterando un bloque latente. Nos distinguimos por el dominio, el barrido de iteraciones y el análisis interno.

3. Método

3.1. Arquitecturas

Ambos modelos comparten el bloque de GPT-2 (Radford et al., 2019): normalización previa, atención multicabeza causal, red *feed-forward* de dimensión $4d$ con GELU y conexiones residuales, $B_\theta(\mathbf{h}) = \mathbf{u} + \mathrm{FFN}(\mathrm{LN}_2(\mathbf{u}))$ con $\mathbf{u} = \mathbf{h} + \mathrm{MHA}(\mathrm{LN}_1(\mathbf{h}))$. La entrada es la suma de los *em-*

beddings de carácter y de posición (aprendido), $\mathbf{h}_0$, y la salida $\mathrm{softmax}(W_o\,\mathrm{LN}(\mathbf{h}_D))$ con D la profundidad efectiva. Un **GPT** de L capas usa bloques distintos, $\mathbf{h}_\ell = B_{\theta_\ell}(\mathbf{h}_{\ell-1})$; un **Looped Transformer** de N iteraciones reutiliza el mismo,

$$\mathbf{h}_t = B_\theta(\mathbf{h}_{t-1}), \qquad t = 1, \ldots, N, \qquad (1)$$

con $D = L$ y $D = N$ respectivamente. Con dimensión d, vocabulario V y contexto T, un bloque tiene $P_B = 12d^2 + 10d$ parámetros y el resto del modelo $P_E = (2V + T)d + 2d$, de modo que $P_{\text{GPT}}(L) = L\,P_B + P_E$ y $P_{\text{Looped}}(N) = P_B + P_E$ no depende de N, mientras que el cómputo por token, $F(D) \approx D\,(24d^2 + 4Td) + 2dV$, crece con D en ambos.

3.2. Implementación y validación

Partimos del código de Raschka (2024), con la atención explícita para devolver sus pesos; ambos modelos exponen la misma interfaz (logits, estados ocultos y atenciones por paso). Comprobamos la corrección con pruebas automáticas: Looped con $N{=}1$ coincide con GPT con $L{=}1$ y mismos pesos (10^{-6}); Looped con $N \in \{2, 4, 8, 12\}$ coincide, en logits y en todos los estados ocultos, con un GPT de N capas con copias del bloque compartido (10^{-5}); el número de parámetros no depende de N y la atención es causal.

3.3. Datos

Cada ejemplo es una historia breve en castellano cuyo nivel es el número de operaciones, de 1 a 6: suma, resta, doble, triple, mitad, tercera y cuarta parte, con números de hasta 20 en el enunciado y respuestas entre 1 y 99. Hay relatos con un protagonista o un recipiente (tipo A) y problemas de comparación entre varias personas (tipo B) cuyas frases pueden estar desordenadas, en estilo detallado o compacto y a partir de plantillas. Un ejemplo de nivel 2 con frases desordenadas:

> *Julia tiene la mitad que Laura. Laura tiene el triple de postales que Pablo. Pablo tiene 18 postales. Entonces, ¿cuántas postales tiene Julia?* → 27

Cada ejemplo guarda su traza de operaciones, con la que se verifica la respuesta, y ninguna pregunta de validación o de test aparece en el entrenamiento. Los niveles 1–4 se reparten en entrenamiento (200 000 ejemplos, 50 000 por nivel), validación (4 000) y test en distribución (ID, 8 000, 2 000 por nivel). Un segundo test de plantillas nuevas (8 000 ejemplos) usa 18 redacciones reservadas que no aparecen en el entrenamiento. Los niveles 5 y 6 (4 000 ejemplos) solo se usan como test fuera de distribución (OOD). Las heurísticas sin razonamiento (respuesta más frecuente, primer o último número) aciertan entre el 0.6 y el 7.6 %.

3.4. Entrenamiento y evaluación

El texto de entrada es `Pregunta: ... \nRespuesta:` seguido del número y de un fin de secuencia, tokenizado por caracteres (97 símbolos); la pérdida se calcula solo sobre la respuesta. Usamos AdamW (Loshchilov and Hutter, 2019) ($\beta = (0.9, 0.95)$, decaimiento 0.1), tasa 10^{-3} con calentamiento de 200 pasos y decaimiento coseno hasta el 10 %, lotes de 64, recorte de gradiente 1.0 y precisión mixta, durante 42 000 pasos y con tres semillas (0, 1 y 2). Evaluamos por generación voraz (hasta 6 caracteres, primer entero extraído) y medimos la exactitud por nivel (coincidencia exacta); en los modelos con bucles variamos las iteraciones en test, N', de 1 a $2N$ sin reentrenar.

4. Experimentos

4.1. Configuraciones y comparaciones controladas

Todos los modelos usan contexto de 512 caracteres, vocabulario de 97 símbolos y $d{=}256$ con 8 cabezas, salvo un GPT de 4 capas con $d{=}128$ y 4 cabezas (contraste C3). Entrenamos GPT con $L \in \{1, 2, 4, 8\}$ capas y Looped Transformers con $N \in \{2, 4, 8\}$ iteraciones, con las semillas 0, 1 y 2 (24 entrenamientos de 42 000 pasos). La Tabla 1 indica el número de parámetros de cada modelo. La matriz permite tres contrastes: **(C1)** *igual cómputo*, Looped N frente a GPT $L{=}N$, con los mismos FLOPs por token y hasta N veces más parámetros el segundo; **(C2)** *iguales parámetros*, Looped N frente a GPT $L{=}1$, que no comparten el cómputo; y **(C3)** *parámetros y profundidad parecidos*, Looped $N{=}4$ (0.97 M de parámetros) frente a ese GPT de 4 capas con $d{=}128$ (0.88 M), que tiene la misma profundidad y casi los mismos parámetros pero 3.2 veces menos FLOPs por token.

4.2. Resultados

En las comparaciones, la primera cifra es siempre la del Looped y la segunda la del GPT.

Igual número de parámetros (C2). Con 0.97 M de parámetros, el Looped Transformer llega a 25.6 y 16.5 % en los niveles 3 y 4 con $N{=}2$, a 47.0

		En distribución				OOD		
Modelo	Parám. (M)	Niv. 1	Niv. 2	Niv. 3	Niv. 4	Niv. 5	Niv. 6	Plant. nuevas
GPT L=1	0.97	87.6±0.6	35.3±0.3	8.9±0.6	6.7±0.6	5.1±0.4	4.1±0.3	16.1±0.8
GPT L=2	1.76	96.8±0.5	73.1±3.5	26.1±3.4	16.9±1.7	12.5±0.9	8.9±0.4	22.4±2.4
GPT L=4	3.34	96.9±0.2	78.7±1.8	48.8±4.1	34.9±4.1	23.9±3.0	14.5±1.8	25.5±1.9
GPT L=8	6.49	97.0±0.2	**80.9**±0.4	54.3±2.1	40.2±1.8	25.4±2.7	13.6±1.7	26.7±0.6
Looped N=2	0.97	96.9±0.4	71.9±0.4	25.6±1.2	16.5±0.2	12.2±0.4	9.0±0.4	21.1±1.7
Looped N=4	0.97	**97.3**±0.2	78.7±0.6	47.0±1.1	31.6±0.3	21.1±2.3	14.0±0.1	23.3±1.0
Looped N=8	0.97	97.1±0.2	79.0±0.2	**57.2**±1.1	**45.8**±1.4	**33.4**±2.2	**21.2**±2.2	**29.8**±4.2
GPT L=4 (d=128)	0.88	96.5±0.5	65.5±7.4	35.0±10.7	24.3±8.7	16.5±5.1	11.3±3.0	22.6±2.5

Tabla 1: Exactitud (%) por nivel, media ± desviación típica sobre tres semillas (niveles 1–4: test en distribución; 5 y 6: fuera de distribución; *Plant. nuevas*: redacciones no vistas). En negrita, la mejor media.

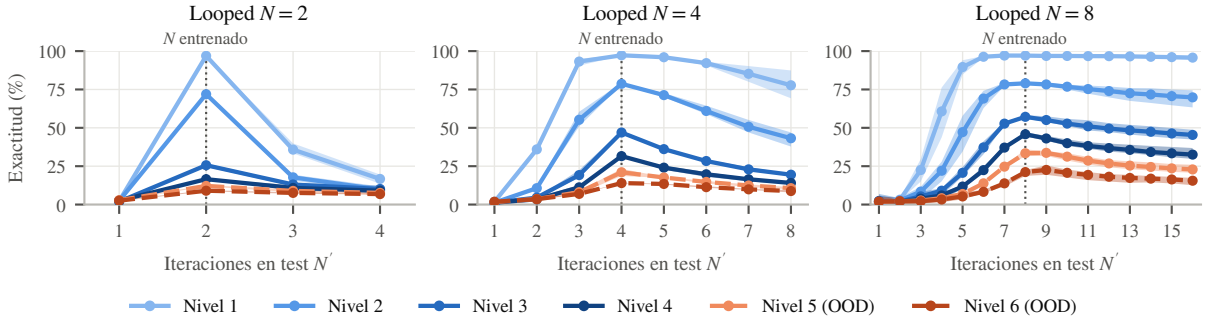


Figura 1: Exactitud de un Looped Transformer entrenado con N iteraciones al evaluarlo con N' sin reentrenar (banda: mínimo y máximo entre semillas).

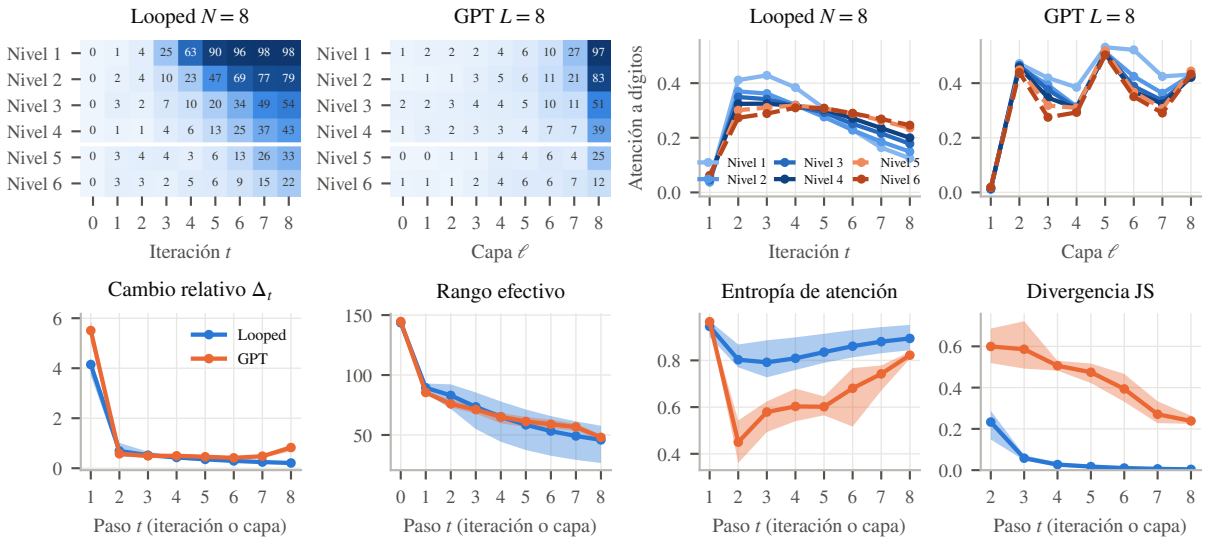


Figura 2: Interior de Looped N=8 y GPT L=8. Arriba: *logit lens* y atención a dígitos (por nivel). Abajo (niveles 1–4; banda: mínimo y máximo entre semillas): cambio relativo Δ_t, rango efectivo, entropía y divergencia JS entre pasos.

y 31.6 % con N=4 y a 57.2 y 45.8 % con N=8 (Tabla 1), mientras que el GPT de una capa apenas resuelve esos niveles (8.9 y 6.7 %, frente al 6.2 y 5.4 % de responder siempre la respuesta más frecuente): iterar el bloque aporta una capacidad que no se obtiene con más parámetros.

Igual cómputo (C1). Con N=2 y N=4 el Looped queda al nivel del GPT de igual profundidad o algo por debajo, dentro de la desviación entre semillas (nivel 4: 16.5 frente a 16.9 % con $N{=}L{=}2$, y 31.6 frente a 34.9 % con $N{=}L{=}4$), con 1.8 y 3.4 veces menos parámetros. Con N=8 iguala o supera

al GPT de 8 capas, que tiene 6.7 veces más (nivel 3: 57.2 frente a 54.3 %; nivel 4: 45.8 frente a 40.2 %). A igual profundidad el tiempo de entrenamiento es el mismo (2.4 frente a 2.5 h con D=8): el bucle ahorra parámetros, no tiempo. En C3, el Looped N=4 supera al GPT con d=128 en los niveles 3 y 4 (47.0 frente a 35.0 % y 31.6 frente a 24.3 %), con mucha menos varianza entre semillas.

Generalización a más operaciones. En los niveles 5 y 6 la ventaja del Looped N=8 es mayor: 33.4 y 21.2 % frente a 25.4 y 13.6 % del GPT de 8 capas, que apenas mejora al de 4 capas (23.9 y 14.5 %) aunque sí lo supera en los niveles 3 y 4. Una posible causa es que, con como máximo 4 operaciones en el entrenamiento, nada obliga a las capas adicionales del GPT a repetir un mismo paso: ganan capacidad para ajustar los niveles 1 a 4, mientras que el bucle debe aprender un paso reutilizable. Los problemas OOD no son más largos que los de entrenamiento (411 caracteres como máximo en ambos), así que no se trata de posiciones no vistas.

Número de iteraciones necesarias (P2). La Fig. 1 varía las iteraciones en test sin reentrenar. Con menos de las entrenadas la exactitud cae con rapidez, y antes cuanto más difícil es el nivel: con N=8, el nivel 1 alcanza 89.6 % con 5 iteraciones y 97.1 % con 7, mientras que el nivel 4 sigue mejorando hasta la octava (37.1 % con 7 y 45.8 % con 8). Las iteraciones necesarias crecen con la dificultad. Con más de las entrenadas la exactitud también baja, y más deprisa cuanto menor es N (N=4 con 8 iteraciones: nivel 1 de 97.3 a 77.7 %), y en los niveles 5 y 6 el máximo está en N o en N+1, por lo que no observamos extrapolación en profundidad.

5. Análisis

Logit lens (P2). Con *teacher forcing* y 200 problemas por nivel ID (100 OOD), proyectamos cada estado de la posición de inicio de la respuesta con la cabeza de salida (*logit lens*, nostalgebraist, 2020), $\hat{y}_t = \arg\max_v [\mathrm{head}(\mathbf{h}_t)]_v$, y llamamos paso de resolución al primer t desde el que la respuesta es correcta hasta el final. En el Looped N=8 la fracción de problemas resueltos crece con las iteraciones, y más tarde cuanto mayor es el nivel: el paso medio de resolución pasa de 4.2 en el nivel 1 a 6.5 en el 4 y a 7.0 en el 6 (Fig. 2, arriba a la izquierda). En el GPT de 8 capas, en cambio, la respuesta casi no aparece en las capas intermedias (27 % con 7 capas en el nivel 1) y surge en la última, sea cual sea el nivel (paso medio 7.6 a 7.8).

Representaciones y atención (P4). En ambos modelos el rango efectivo (Roy and Vetterli, 2007), que mide cuántas direcciones usa el estado, baja de unos 145 a unos 46. En el Looped el cambio relativo Δ_t se reduce de 4.1 a 0.2, pero la norma crece de 2.5 a 61.5, así que no hay evidencia de un punto fijo. Su atención es casi la misma en cada iteración (divergencia JS de 0.23 entre las dos primeras y de 0.01 o menos desde la sexta), mientras que cada capa del GPT atiende de forma distinta (JS de 0.24 a 0.60). La atención de la posición de respuesta a los dígitos del enunciado decrece en el Looped cuando la respuesta ya está formada (nivel 1: de 0.43 a 0.12) y menos en los niveles difíciles (nivel 6: de 0.31 a 0.25).

Tipos de problema y plantillas nuevas. En los modelos de profundidad 4 y 8 y en los niveles 3 y 4, los problemas de tipo B con frases desordenadas se resuelven solo en un 12–14 %, frente al 36–47 % del tipo B ordenado y al 53–74 % del tipo A (relatos de un protagonista o un recipiente). Con redacciones nuevas la exactitud cae de 64–70 % a 23–30 %: los modelos dependen de las plantillas vistas.

6. Conclusiones y limitaciones

Un único bloque iterado ocho veces (0.97 M de parámetros) iguala o supera a un GPT de ocho capas (6.5 M) en los niveles difíciles y, sobre todo, fuera de la distribución, y con los mismos parámetros mejora con claridad al GPT de una capa (P1, P3). Con dos y cuatro iteraciones queda al nivel del GPT de igual profundidad o algo por debajo. Las iteraciones necesarias crecen con la dificultad (P2) y, con menos o con más de las entrenadas, la exactitud cae: no hay extrapolación en profundidad. Por dentro, el Looped resuelve más problemas a medida que itera, y más tarde cuanto mayor es el nivel, con casi la misma atención en cada iteración, mientras que el GPT los resuelve en la última capa (P4).

Limitaciones. Modelos pequeños y datos sintéticos. Los niveles 1 y 2 miden menos razonamiento nuevo: el 100 % de sus trazas de test ya aparece en el entrenamiento con otro texto. Hiperparámetros comunes, sin ajuste por profundidad.

References

Mostafa Dehghani, Stephan Gouws, Oriol Vinyals, Jakob Uszkoreit, and Łukasz Kaiser. 2019. Universal transformers. In *International Conference on Learning Representations (ICLR)*.

Jonas Geiping, Sean McLeish, Neel Jain, John Kirchenbauer, Siddharth Singh, Brian R. Bartoldson, Bhavya Kailkhura, Abhinav Bhatele, and Tom Goldstein. 2025. Scaling up test-time compute with latent reasoning: A recurrent depth approach. *arXiv preprint arXiv:2502.05171*.

Angeliki Giannou, Shashank Rajput, Jy-yong Sohn, Kangwook Lee, Jason D. Lee, and Dimitris Papailiopoulos. 2023. Looped transformers as programmable computers. In *International Conference on Machine Learning (ICML)*.

Ilya Loshchilov and Frank Hutter. 2019. Decoupled weight decay regularization. In *International Conference on Learning Representations (ICLR)*.

nostalgebraist. 2020. Interpreting GPT: the logit lens. LessWrong.

Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. 2019. Language models are unsupervised multitask learners. OpenAI technical report.

Sebastian Raschka. 2024. *Build a Large Language Model (From Scratch)*. Manning.

Olivier Roy and Martin Vetterli. 2007. The effective rank: A measure of effective dimensionality. In *European Signal Processing Conference (EUSIPCO)*.

Nikunj Saunshi, Nishanth Dikkala, Zhiyuan Li, Sanjiv Kumar, and Sashank J. Reddi. 2025. Reasoning with latent thoughts: On the power of looped transformers. In *International Conference on Learning Representations (ICLR)*.